

# 从 MHA 到 GQA：大模型训练优化实验

基于 nanoGPT，用一个下午跑通五组对比实验，搞清楚现代 LLM 为什么长这样

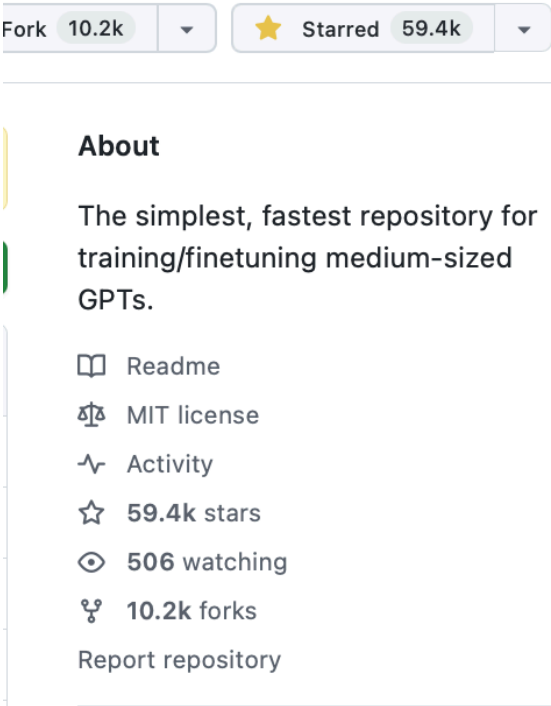
## 1. 为什么要做这个实验

如果你看一眼 LLaMA、Qwen、DeepSeek 这些主流大模型的技术报告，会发现它们的架构出奇地相似：都用 GQA（分组查询注意力）、都用 RMSNorm、都用 SiLU 激活函数。令人好奇的是，这些选择并不是一开始就确定的——GPT-2 用的是 MHA + LayerNorm + GELU。从 GPT-2 到 LLaMA，中间发生了什么？

本文试图用一个可控的小实验来回答这个问题。我们在 nanoGPT（Andrej Karpathy 开源的一个极简 GPT 实现）上，逐一替换这三个组件，观察每一步对模型质量的影响。

## 2. 工具箱：nanoGPT 与 WikiText-103

nanoGPT 是目前最流行的 GPT 教学实现，代码量极少——模型定义（`model.py`）约 300 行，训练循环（`train.py`）约 300 行——但功能完整，能在 OpenWebText 上完整复现 GPT-2 124M 的训练过程 [7]。更重要的是，它没有框架包袱，每一行都可以直接改动。



我们的实验跑在 WikiText-103 上，一个约 1 亿 token 的英文维基百科语料。之所以选它，是因为比莎士比亚（1MB）大两个数量级，足够让模型学到有意义的语言模式，又比 OpenWebText（9B）小两个数量级，一个实验十来分钟就能出结果——刚好适合做系统的对照实验。

模型规模统一控制在约 5500 万参数：8 层 Transformer、12 个注意力头、embedding 维度 768、上下文长度 2048。所有实验使用相同的训练配置（500 步、cosine 学习率衰减、AdamW 优化器、

bfloat16 混合精度），唯一的变量就是我们想测试的那个组件。

### 3. 注意力机制：MHA → GQA → MQA

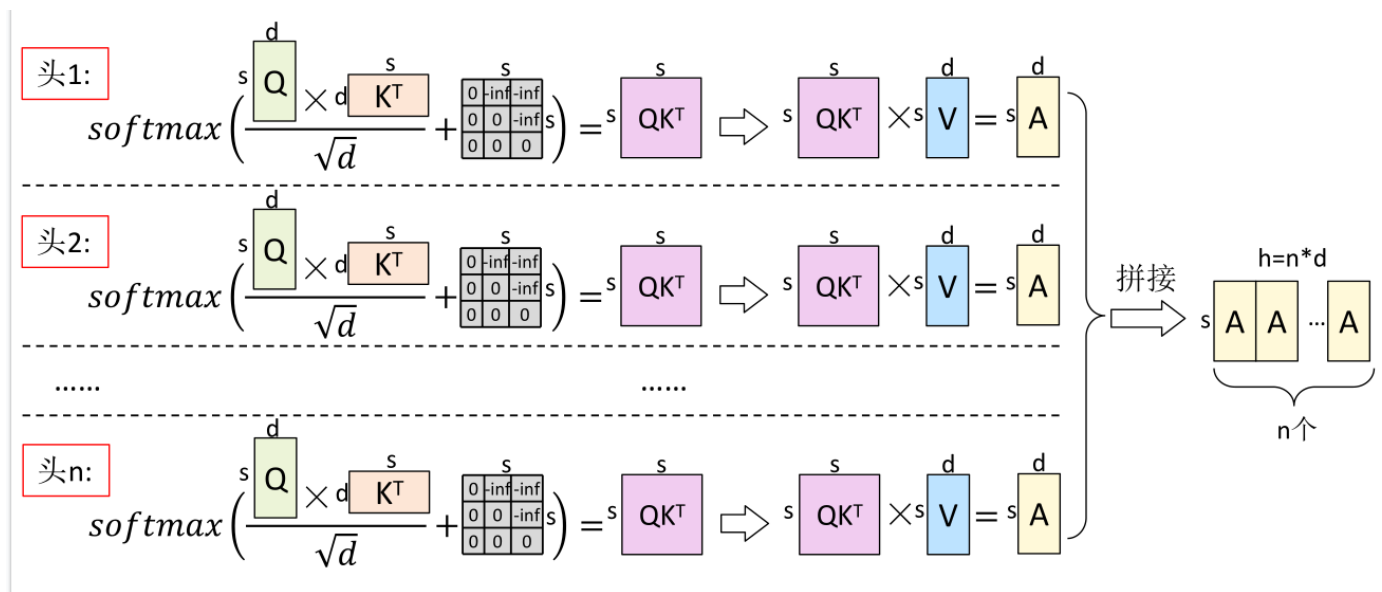
#### 3.1 标准多头注意力（MHA）

自注意力的核心操作：序列中每个 token 发出一个"查询"向量（Query），向所有前置 token 的"键"向量（Key）做相似度匹配，得到的权重用于聚合"值"向量（Value）。写成基本公式：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中除以  $\sqrt{d_k}$  是为了防止点积值过大导致 softmax 梯度消失。

**多头注意力（MHA）** 将这个操作并行执行多次，每个"头"拥有独立的 Q、K、V 投影[1]：



投影的数学形式：

$$Q_i = xW_Q^i, \quad K_i = xW_K^i, \quad V_i = xW_V^i, \quad \text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{MHA}(x) = \text{Concat}(\text{head}_1, \dots, \text{head}_n) W_O$$

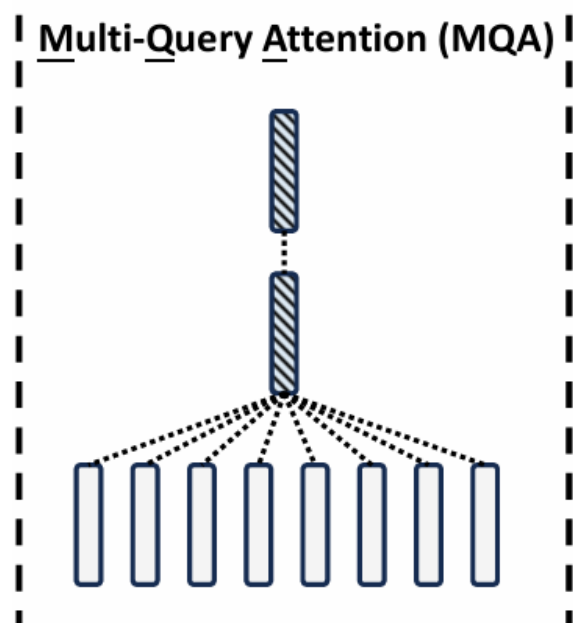
参数量：Q、K、V 投影各为  $d_{\text{model}} \times d_{\text{model}}$ ，共  $3d_{\text{model}}^2$ ，输出投影  $W_O$  另计  $d_{\text{model}}^2$ 。

以 LLaMA2 为例感受一下 MHA 的规模：7B 版本有  $d_{\text{model}} = 4096$ ， $n_{\text{head}} = 32$ ， $d_k = 128$ ，70B 版本则翻倍至  $d_{\text{model}} = 8192$ ， $n_{\text{head}} = 64$ ， $d_k = 128$ 。注意力层的参数量随模型规模线性增长。

MHA 在推理时面临一个实际问题——**KV cache**。自回归生成每一步都要重新计算所有历史 token 的 K 和 V，为免重复运算，通常将已计算的 K、V 缓存起来。但随着序列变长，KV cache 的显存占用线性增长：缓存大小 =  $2 \times n_{\text{head}} \times d_k \times L_{\text{seq}}$ 。对于长序列推理，这会迅速成为瓶颈。

#### 3.2 多查询注意力（MQA）

最激进的解决方案是 2019 年 Shazeer 提出的 **MQA**（Multi-Query Attention）[3]：所有 Q 头共享同一对 K 和 V。

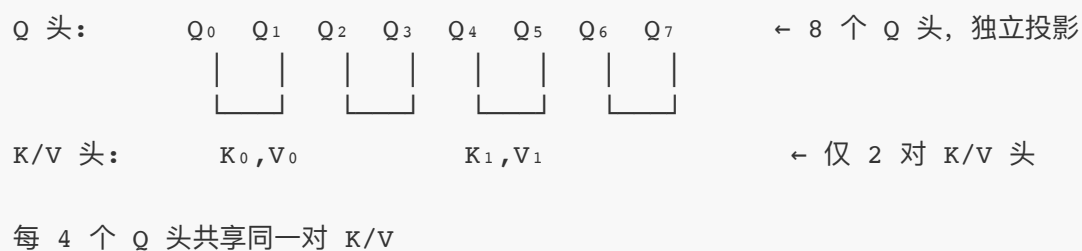


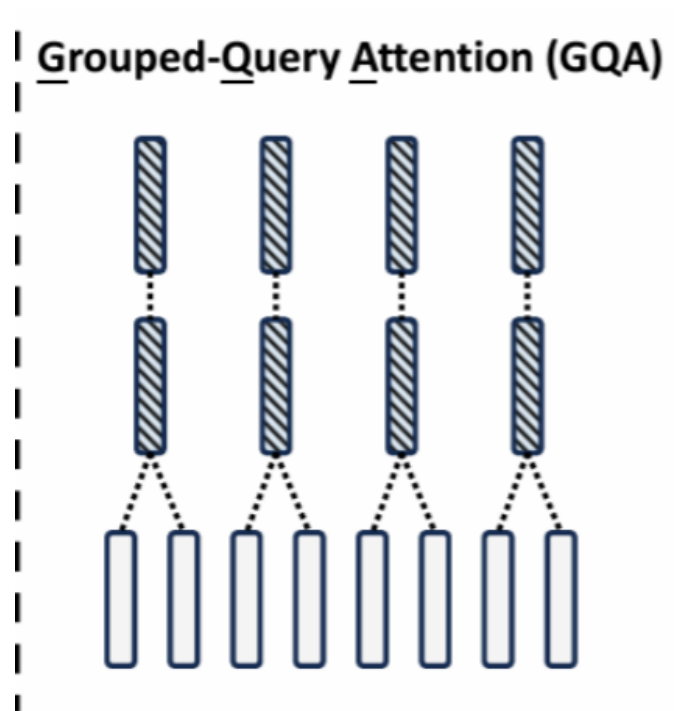
所有注意力头的 Q 各自独立，但 K 和 V 只有一份。KV cache 直接缩减为原来的  $1/n_{head}$ 。代价是 K、V 的表达能力明显受限，模型质量和训练稳定性会受到一定影响。

### 3.3 分组查询注意力（GQA）

MHA 的问题是推理开销。生成 token 时，每一步都需要缓存所有历史的 K 和 V（即 KV cache）， $n_{head}$  组完整的 K、V 矩阵占用了大量显存。2019 年，Shazeer 发现了一个反直觉的事实：即使所有注意力头共享同一对 K 和 V（即 MQA），模型质量下降也非常有限[3]。

这启发 Ainslie 等人提出了 **GQA**[2]：将 Q 头分成若干组，每组共享一对 K/V 头。以  $n_{head} = 8, n_{kv\_head} = 2$  为例，用图表示：





投影过程：

$$Q = xW_Q \in \mathbb{R}^{B \times T \times n_{head} \cdot d_h}$$

$$K = xW_K \in \mathbb{R}^{B \times T \times n_{kv\_head} \cdot d_h} \quad (\text{更小})$$

$$V = xW_V \in \mathbb{R}^{B \times T \times n_{kv\_head} \cdot d_h} \quad (\text{更小})$$

计算注意力前，将 K、V 广播到所有 Q 头：

$$K \leftarrow K.\text{repeat\_interleave}\left(\frac{n_{head}}{n_{kv\_head}}\right)$$

三种机制的关系总结：

| 机制  | Q 头数       | K/V 头数         | KV 参数量                    | 说明   |
|-----|------------|----------------|---------------------------|------|
| MHA | $n_{head}$ | $n_{head}$     | $2d_{model}^2$            | 每头独立 |
| GQA | $n_{head}$ | $n_{kv\_head}$ | $n_{kv\_head} \cdot 2d^2$ | 分组共享 |

|     |            |                |                                           |      |
|-----|------------|----------------|-------------------------------------------|------|
| GQA | $n_{head}$ | $n_{kv\_head}$ | $\frac{1}{n_{kv\_head}} \cdot 2d_{model}$ | 万组共享 |
| MQA | $n_{head}$ | 1              | $\frac{1}{n_{head}} \cdot 2d_{model}^2$   | 全部共享 |

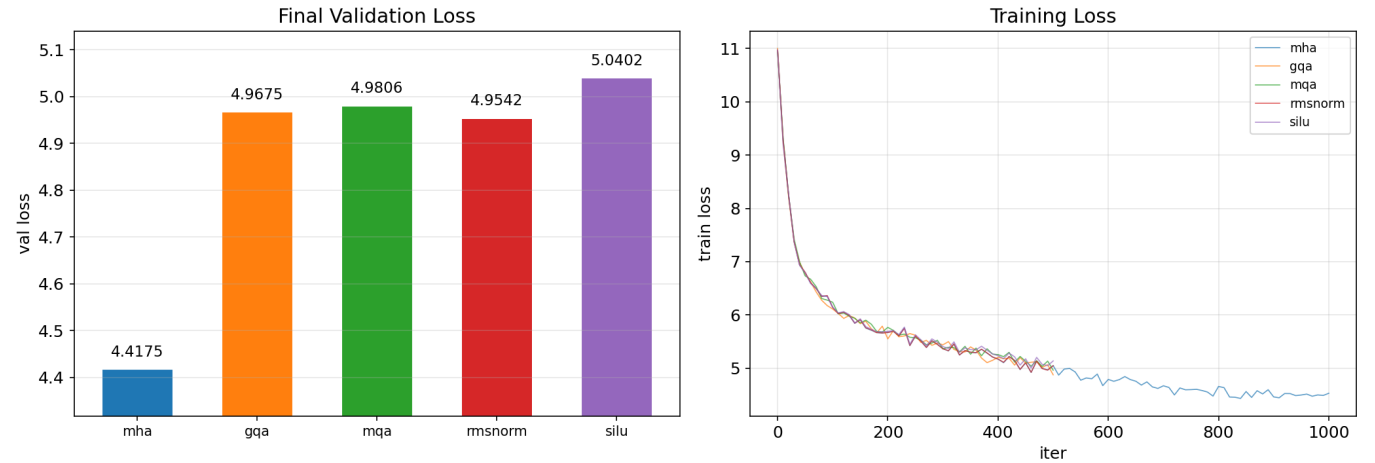
GQA 的工程收益十分明确：KV cache 大小正比于 KV 头数。当  $n_{head} = 12$ ,  $n_{kv\_head} = 3$  时，推理显存中的 KV cache 直接缩减到原来的四分之一，这对长序列场景至关重要。

### 3.4 实验结果

我们在相同配置下分别训练了 MHA、GQA (n\_kv=3) 和 MQA (n\_kv=1) 三个模型：

| 指标       | MHA          | GQA          | MQA          |
|----------|--------------|--------------|--------------|
| KV 头数    | 12           | 3            | 1            |
| KV 参数占比  | 100%         | 25%          | 8.3%         |
| 单步耗时     | 265ms        | 258ms        | 258ms        |
| 验证集 loss | <b>4.954</b> | <b>4.968</b> | <b>4.981</b> |
| 相对 MHA   | —            | +0.014       | +0.027       |

最引人注目的是 GQA：KV 参数减少了 75%，验证集 loss 仅增加了 0.014。这是一个几乎可以忽略的代价。即使是最激进的 MQA（所有头共享 K/V），loss 也不过增加了 0.027。这组数据清晰地解释了为什么 GQA 成为了现代 LLM 的标配——它在训练质量和推理效率之间找到了最佳的平衡点。



## 4. 归一化：LayerNorm vs RMSNorm

归一化层位于每个子层（注意力和 MLP）之前，将输入张量归一化。LayerNorm 使用均值和标准差的估计值来归一化输入，而 RMSNorm 只使用均值的估计值。

归一化层位于每个子层（注意是和 MLP）之前，确保输入数值稳定。**LayerNorm** 先减均值再除标准差，然后做仿射变换：

$$\text{LayerNorm}(x) = \gamma \cdot \frac{x - \mu}{\sigma} + \beta, \quad \mu = \frac{1}{d} \sum x_i, \quad \sigma = \sqrt{\frac{1}{d} \sum (x_i - \mu)^2 + \epsilon}$$

**RMSNorm** 在 2019 年由 Zhang 和 Sennrich 提出，只做了一个改动——去掉均值中心化[4]：

$$\text{RMSNorm}(x) = \gamma \cdot \frac{x}{\text{rms}(x)}, \quad \text{rms}(x) = \sqrt{\frac{1}{d} \sum x_i^2 + \epsilon}$$

|       | LayerNorm       | RMSNorm  |
|-------|-----------------|----------|
| 均值中心化 | 需要              | 不需要      |
| 可学习参数 | $\gamma, \beta$ | $\gamma$ |
| 计算步骤  | 4 步             | 3 步      |

省掉均值这一步，不仅减少了计算量，还去掉了偏置参数  $\beta$ 。LLaMA 系列从第一代就采用了 RMSNorm。

## 实验结果

| 指标       | LayerNorm | RMSNorm |
|----------|-----------|---------|
| 验证集 loss | 4.954     | 4.954   |
| 单步耗时     | 265ms     | 264ms   |

在我们的实验中，两者的表现几乎完全一致（loss 差距小于 0.001）。RMSNorm 以更简单的计算实现了相同的效果，印证了 LLaMA 团队的工程判断。

## 5. 激活函数：GELU vs SiLU

GPT-2 使用的 GELU（Gaussian Error Linear Unit）是对 ReLU 的重要改进——在零附近平滑过渡，避免 ReLU 在负半轴完全“死掉”[6]：

$$\text{GELU}(x) \approx 0.5x \left( 1 + \tanh \left[ \sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right] \right)$$

SiLU（也叫 Swish）则更加简洁，直接用 sigmoid 函数替代了复杂的累积分布函数近似[5]：

$$\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

两者形状相似，但 SiLU 在正值区域永远不会饱和（导数始终非零），且计算上少了 tanh 近似这一环。LLaMA 选择 SiLU 的理由之一正是计算效率。

### 实验结果

| 指标       | GELU  | SiLU  |
|----------|-------|-------|
| 验证集 loss | 4.954 | 5.040 |
| 单步耗时     | 265ms | 264ms |

SiLU 在我们的实验中明显弱于 GELU（loss 高出 0.086）。这个结果与直觉不完全一致，但可能反映了一个重要事实：SiLU 的优势可能需要在更大规模上才能体现。55M 参数、500 步训练的设定下，GELU 的小模型适配性更好。这一"反直觉"结果本身也有价值——它提醒我们，大模型上的经验不一定直接适用于小模型。

## 6. 五组实验全貌

把所有结果放在一张表里，结论一目了然：

| 实验              | 变量       | val loss | vs baseline |
|-----------------|----------|----------|-------------|
| MHA + LN + GELU | baseline | 4.954    | —           |
| GQA (n_kv=3)    | 注意力      | 4.968    | +0.014      |
| MQA (n_kv=1)    | 注意力      | 4.981    | +0.027      |
| RMSNorm         | 归一化      | 4.954    | ~0          |
| SiLU            | 激活函数     | 5.040    | +0.086      |

## 7. 讨论

### 三个明确的结论

第一，**GQA** 是目前注意力机制的最优解。用四分之一的 KV 参数换来了几乎无损的质量，这一结论在 55M 小模型上依然成立。考虑到长序列推理时 KV cache 是主要瓶颈，GQA 带来的四倍缓存压缩是实打实的工程收益。

第二，**RMSNorm** 可以放心替换 **LayerNorm**。无论是训练质量还是速度，两者在本实验中都无法区分。RMSNorm 的简洁性（少一个 bias、少一次均值计算）使其成为更优的默认选择。

第三，激活函数的选择在小模型上仍需谨慎。SiLU 的表现低于预期，暗示大模型上的成功经验未必能线性外推到小模型。这一发现对边缘设备上的小语言模型设计有参考意义。

## 本实验的局限

55M 参数和 500 步训练显然不是大模型训练的真实图景。部分结论——尤其是 SiLU 的相对表现——可能需要更大规模验证。此外，我们只用了困惑度（perplexity）作为评价指标，没有涉及下游任务（如分类、推理、对话），对实际应用场景的参考价值有限。不过这个规模的实验已经足以展示几个关键组件的相对优劣。

## 8. 结语

回到最初的问题：从 GPT-2 到 LLaMA，架构变化背后有充分的理由。GQA 在几乎不牺牲质量的前提下大幅降低了推理成本，RMSNorm 以更简单的方式实现了等价效果，这两点在我们的实验中得到了清晰的验证。SiLU 的选择则可能更多地依赖于大模型规模下的收益，在小模型场景中需要更谨慎的评估。

这些结果也说明了一个方法论上的要点：大模型架构中的每一个"默认选择"，背后都经历了大量的消融实验。用 nanoGPT 这样的小工具复现这些实验，不仅能加深理解，也能培养对"为什么大模型长这样"的直觉。

## 实验框架

```
src/
├── model.py           ← GPT 模型定义，含 Attention / Norm / MLP 工厂
├── train.py           ← 训练循环 + 日志
├── configurator.py    ← 命令行参数覆盖配置
├── sample.py          ← 模型采样生成
└── bench.py           ← 性能基准测试

config/
├── train_wikitext103.py      ← MHA baseline
├── train_wikitext103_gqa.py  ← GQA (n_kv=3)
├── train_wikitext103_mqa.py  ← MQA
├── train_wikitext103_rmsnorm.py ← RMSNorm
└── train_wikitext103_silu.py  ← SiLU

data/wikitext103/
├── prepare.py              ← 数据预处理 (parquet → .bin)
├── train.bin                ← 训练数据 (~100M tokens)
├── val.bin                  ← 验证数据
└── meta.pkl                 ← 词表信息

out-*/log_*.txt             ← 各实验的训练日志 + loss 记录
plot_results.py             ← 实验结果画图
results_all.png              ← 五组对比图
```

---

实验流程: `python data/wikitext103/prepare.py` → `python src/train.py`  
`config/<config>.py` → `python plot_results.py out-*`

github开源链接: <https://github.com/Mutsumimimi/nanoGPT/tree/dev>

## 参考文献

---

1. Vaswani et al., "Attention Is All You Need", NeurIPS 2017
2. Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints", EMNLP 2023
3. Shazeer, "Fast Transformer Decoding: One Write-Head Is All You Need", 2019
4. Zhang & Sennrich, "Root Mean Square Layer Normalization", NeurIPS 2019
5. Elfwing et al., "Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning", 2018
6. Hendrycks & Gimpel, "Gaussian Error Linear Units (GELUs)", 2018
7. Karpathy, "nanoGPT", <https://github.com/karpathy/nanoGPT>